Date: 2 March 2026

Batch: D3 **Roll No.: 16010125814**

Experiment No. 04

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of the Staff In-charge with date

TITLE: CPU Scheduling – Non Preemptive

AIM: Implementation of Basic CPU Scheduling Algorithms – Non Preemptive [FCFS , SJF]

Expected Outcome of Experiment:

CO 2 Illustrate and analyse the Process, threads, process scheduling and thread scheduling

---

Books/ Journals/ Websites referred:

1. **Silberschatz A., Galvin P., Gagne G. "Operating Systems Principles", Willey Eight edition.**
2. **Achyut S. Godbole , Atul Kahate "Operating Systems" McGraw Hill Third Edition.**
3. **William Stallings, "Operating System Internal & Design Principles", Pearson.**
4. **Andrew S. Tanenbaum, "Modern Operating System", Prentice Hall.**

Pre Lab/ Prior Concepts:

Most systems handle numerous processes with short CPU bursts interspersed with I/O requests and a few processes with long CPU bursts. To ensure good time-sharing performance, a running process may be preempted to allow another to run. The ready list, or run queue, maintains all processes ready to run and not blocked by I/O or other system requests. Entries in this list point to the process control block, which stores all process information and state.

When an I/O request completes, the process moves from the waiting state to the ready state and is placed on the run queue. The process scheduler, a key component of the

operating system, decides whether the current process should continue running or if another should take over. This decision is triggered by four events:

1. The current process issues an I/O request or system request, moving it from running to waiting.
2. The current process terminates.
3. A timer interrupt indicates the process has run for its allotted time, moving it from running to ready.
4. An I/O operation completes, moving the process from waiting to ready, potentially preempting the current process.

The scheduling algorithm, or policy, determines the sequence and duration of process execution, a complex task given the limited information about ready processes.

Description of the application to be implemented:

First-Come, First-Served Scheduling:

First-Come, First-Served (FCFS) is a simple CPU scheduling algorithm where processes are executed in the order they arrive in the ready queue. The process that arrives first is given priority and is executed first, followed by the next process in line. It is a non-preemptive algorithm, meaning once a process starts executing, it runs to completion. While FCFS is easy to implement, it may suffer from the convoy effect, where long processes delay shorter ones.

Shortest job first :

Shortest Job First (SJF) is a CPU scheduling algorithm that selects the process with the smallest burst time to execute next. It can be preemptive (Shortest Remaining Time First - SRTF) or non-preemptive. SJF aims to minimize average waiting time and turnaround time by prioritizing shorter tasks. However, it requires knowledge of the burst time in advance and can lead to starvation for longer processes if short jobs keep arriving.

Implementation details: (printout of code)

1. FCFS

```c
#include <stdio.h>
int main()
{
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n], p[n];

    for(int i = 0; i < n; i++)
    {
        p[i] = i + 1;

        printf("\nEnter Arrival Time for P%d: ", i+1);
        scanf("%d", &at[i]);

        printf("Enter Burst Time for P%d: ", i+1);
        scanf("%d", &bt[i]);
    }
    for(int i = 0; i < n-1; i++)
    {
        for(int j = 0; j < n-i-1; j++)
        {
            if(at[j] > at[j+1])
            {
                int temp = at[j];
                at[j] = at[j+1];
                at[j+1] = temp;

                temp = bt[j];
                bt[j] = bt[j+1];
                bt[j+1] = temp;

                temp = p[j];
                p[j] = p[j+1];
                p[j+1] = temp;
            }
        }
    }
    ct[0] = at[0] + bt[0];
    for(int i = 1; i < n; i++)
    {
        if(ct[i-1] < at[i])
```

```
        printf("Enter Burst Time for P%d: ", i+1);
        scanf("%d", &bt[i]);
    }
    for(int i = 0; i < n-1; i++)
    {
        for(int j = 0; j < n-i-1; j++)
        {
            if(at[j] > at[j+1])
            {
                int temp = at[j];
                at[j] = at[j+1];
                at[j+1] = temp;

                temp = bt[j];
                bt[j] = bt[j+1];
                bt[j+1] = temp;

                temp = p[j];
                p[j] = p[j+1];
                p[j+1] = temp;
            }
        }
    }
    ct[0] = at[0] + bt[0];
    for(int i = 1; i < n; i++)
    {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }
    for(int i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    return 0;
}
```

```
kjsce@kjsce-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~$ gcc fcfs1.c -o fcfs1
kjsce@kjsce-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~$ ./fcfs1
Enter number of processes: 3

Enter Arrival Time for P1: 0
Enter Burst Time for P1: 4

Enter Arrival Time for P2: 2
Enter Burst Time for P2: 4

Enter Arrival Time for P3: 2
Enter Burst Time for P3: 3


Process AT      BT      CT      TAT     WT
P1      0       4       4       4       0
P2      2       4       8       6       2
P3      2       3       11      9       6
kjsce@kjsce-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~$
```

2. SJF

```
#include <stdio.h>

struct process {
    int pid, at, bt, wt, tat, ct, completed;
};

int main() {
    struct process p[4];
    int i, completed = 0, current_time = 0;
    int shortest, min_bt;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter Arrival Time and Burst Time (in ms):\n");
    for(i = 0; i < 4; i++) {
        p[i].pid = i + 1;
        printf("P%d AT: ", i+1);
        scanf("%d", &p[i].at);
        printf("P%d BT: ", i+1);
        scanf("%d", &p[i].bt);
        p[i].completed = 0;
    }

    while(completed < 4) {
        shortest = -1;
        min_bt = 9999;

        for(i = 0; i < 4; i++) {
            if(p[i].at <= current_time &&
               p[i].completed == 0 &&
               p[i].bt < min_bt) {

                min_bt = p[i].bt;
                shortest = i;
            }
        }

        if(shortest == -1) {
            current_time++;
        } else {
            p[shortest].wt = current_time - p[shortest].at;
            current_time += p[shortest].bt;
            p[shortest].ct = current_time;
            p[shortest].tat = p[shortest].ct - p[shortest].at;
            p[shortest].completed = 1;
```

```c
        } else {
            p[shortest].wt = current_time - p[shortest].at;
            current_time += p[shortest].bt;
            p[shortest].ct = current_time;
            p[shortest].tat = p[shortest].ct - p[shortest].at;
            p[shortest].completed = 1;

            avg_wt += p[shortest].wt;
            avg_tat += p[shortest].tat;
            completed++;
        }
    }

    printf("\nPID\tAT\tBT\tWT\tTAT\n");
    for(i = 0; i < 4; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].wt, p[i].tat);
    }

    printf("\nAverage WT = %.2f ms", avg_wt/4);
    printf("\nAverage TAT = %.2f ms\n", avg_tat/4);

    return 0;
}
```

```
kjsce@kjsce-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~$ gcc sjf1.c -o sjf1
kjsce@kjsce-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~$ ./sjf1
Enter Arrival Time and Burst Time (in ms):
P1 AT: 2
P1 BT: 3
P2 AT: 5
P2 BT: 4
P3 AT: 2
P3 BT: 4
P4 AT: 5
P4 BT: 1

PID     AT      BT      WT      TAT
P1      2       3       0       3
P2      5       4       1       5
P3      2       4       8       12
P4      5       1       0       1

Average WT = 2.25 ms
Average TAT = 5.25 ms
kjsce@kjsce-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~$
```

Conclusion :

Hence, after implementing this experiment, we understood and successfully implemented two fundamental non-preemptive CPU scheduling algorithms: FCFS and SJF. We analyzed their impact on process execution, waiting time, and overall system performance.

Multiple-Choice Questions (MCQs)

1. **In FCFS scheduling, the process that arrives first:**

   A) Gets executed first

   B) Gets executed last

   C) Has the highest priority

   D) Has the shortest burst time

2. **Which scheduling algorithm can cause the "Convoy Effect"?**

   A) FCFS

   B) SJF

   C) Round Robin

   D) Priority Scheduling

3. **SJF scheduling algorithm is also known as:**

   A) Shortest Remaining Time First

   B) Shortest Time Next

   C) Longest Job First

Post Lab Descriptive Questions

1. Compare FCFS and SJF.

Aspect	FCFS (First-Come, First-Served)	SJF (Shortest Job First)
Execution Order	Processes are executed in the order of their arrival.	Processes with the shortest burst time are executed first.
Efficiency	Can be inefficient, especially with a mix of long and short processes (convoy effect).	More efficient, reduces waiting time by prioritizing shorter jobs.
Average Waiting Time	Generally results in high average waiting time due to long processes blocking short ones.	Typically results in lower average waiting time due to shorter jobs getting executed first.
Starvation	No starvation, as processes are executed in the order they arrive.	Can cause starvation for longer processes if many short ones keep arriving.

2. Discuss the impact of SJF scheduling on the average waiting time of processes.

   The Shortest Job First (SJF) scheduling algorithm typically reduces the average waiting time of processes because it prioritizes the execution of processes with the shortest burst time. By executing shorter jobs first, SJF minimizes the amount of time longer processes need to wait in the queue. However, SJF can lead to starvation for longer processes, especially in a system with many shorter processes, as they might be continuously delayed. Overall, while SJF improves efficiency and reduces waiting time in many cases, it can be unfair to processes with longer burst times.

3. Consider a set of 4 processes with their respective arrival and burst times given in milliseconds.

Process	Arrival Time	Burst Time
P0	0	20

P1	15	25
P2	30	10
P3	45	15

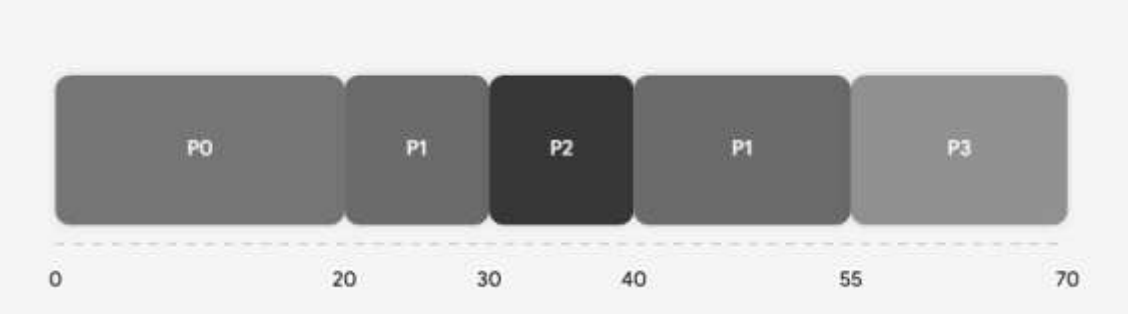
A. **Apply the First-Come, First-Served (FCFS) and Shortest Job First (SJF) scheduling algorithms.**

B. **For each scheduling method:**
- Draw the Gantt Chart.
- Calculate the Average Turnaround Time.
- Calculate the Average Waiting Time.

For FCFS

Average Turnaround Time: 100 / 4 = 25
Average Waiting Time: 30 / 4 = 7.5

For SJF



Average Turnaround Time: 100 / 4 = 25
Average Waiting Time: 30 / 4 = 7.5
Same as FCFS

C. **Analyze the results and comment on the performance of FCFS and SJF for the given process set.**

For this specific set of processes, FCFS and Non-Preemptive SJF perform identically. This happens because at every scheduling decision point (when the CPU becomes idle), the process with the shortest burst time also happened to be

the one that arrived first. For example, at t=45, both P2 and P3 are waiting. SJF picks P2 because it is shorter (10 < 15), while FCFS picks P2 because it arrived earlier (30 < 45). The resulting queue order is unchanged.However, if we apply Preemptive SJF (SRTF), the performance improves significantly. The Average Waiting Time drops from 7.5 ms to 6.25 ms. This is because preemption allows the short process (P2) to interrupt the long process (P1) the moment it arrives at t=30, preventing P2 from being trapped behind P1's massive 25ms burst time.

Date: 02/03/2026 **Signature of faculty in-charge**